FormFix AI

Comprehensive System Architecture, Module Interconnection & Technical Report

CORE ARCHITECTURAL MISSION: Document → Understanding → Action

Generic PDF summarizers fail because they output passive paragraphs of text that students still have to interpret. FormFix AI is purpose-built to eliminate administrative confusion by extracting: (1) Urgent Deadlines, (2) Itemized Required Documents, (3) Translated Bureaucratic Jargon, and (4) An Interactive, Checkable Action Plan.

1. Executive Summary & Problem Context

Every academic year, students, job seekers, and citizens must interact with complicated institutional paperwork—scholarship renewal notices, college admission guidelines, government circulars, and application forms. These documents are notoriously verbose, written in rigid legalistic language, and feature critical requirements (income limits, minimum academic percentages, backlog disqualifications, and cutoff deadlines) buried deep inside lengthy paragraphs. Missing a single supporting certificate or misinterpreting a deadline often causes disqualification, revocation of financial aid, or missed admission cycles.

Why Existing AI Tools Fall Short: General LLM chatbots (e.g. asking a generic model to 'summarize this PDF') produce long text summaries that do not clearly tell the user what physical actions to take next. Furthermore, standard models are prone to hallucinating plausible requirements or fabricating deadlines when dates are missing from the text.

The FormFix Solution: FormFix AI bridges the gap between passive reading and active execution. It processes documents through a multi-stage hybrid pipeline, identifies the physical paperwork required, clarifies complex institutional terms, highlights deadlines with urgency badges, and synthesizes an interactive task checklist that the user can track in real time.

2. Exhaustive Technology Stack & Design Justifications

The technology stack was chosen specifically for maximum execution speed, zero architectural bloat, and ironclad reliability:

Component	Technology & Version	Exact Role in the Application	Why This Specific Tool Was Chosen
Frontend Framework	Streamlit >= 1.35.0	Renders the entire web application, file uploader, loading indicators, interactive checkboxes, and grounded chat.	Enables stateful, reactive UI in pure Python. Eliminates Node/React dependencies and API glue code.
PDF Text Extraction	PyMuPDF (fitz) >= 1.24.0	Opens PDF byte streams in memory, extracts UTF-8 text per page, and checks character density.	High-speed C-based engine. 15-20x faster than PyPDF2/PDFMiner; preserves natural reading order.
Image Preprocessing	Pillow (PIL) >= 10.0.0	Converts images to 8-bit grayscale and applies 1.8x contrast enhancement prior to OCR.	Sharpening edges and eliminating background noise dramatically improves OCR text recognition accuracy.
Optical Character Recognition	pytesseract >= 0.3.10	Extracts text from scanned photocopies, smartphone photos, and image-based PDFs.	Industry-standard open OCR engine with automatic binary path discovery across Windows and Linux.
AI Reasoning & LLM	Groq Cloud API >= 0.9.0 (Qwen 27B / Llama 3)	Executes zero-shot structured JSON document extraction and powers the grounded Q&A; chat assistant.	Groq's LPUs provide sub-second inference (300+ tokens/sec). Guaranteed JSON schema compliance.
Configuration & Secrets	python-dotenv & Streamlit Secrets	Manages secure API credentials across local development (.env) and cloud production (st.secrets).	Multi-tier resolution guarantees zero hardcoded keys and seamless deployment without code edits.
State Management	Streamlit Session State	Maintains task checklist checkbox states, chat message history, and active document buffers.	Runs entirely in server memory. Zero database dependencies; ensures complete user data privacy.

3. Complete Directory Structure & File-by-File Breakdown

The project follows a clean, modular structure where presentation, document processing, optical recognition, and AI reasoning are strictly separated:

FormFix-AI/app.py (Main Application & Presentation Layer):

This is the entry point executed by Streamlit (`streamlit run app.py`). It handles: (1) Custom CSS injection for clean typography, hero headers, and mobile responsiveness; (2) Stateful file upload handling for PDF, PNG, JPG, and JPEG; (3) Dynamic multi-step progress animations during analysis; (4) Rendering the Results Dashboard (deadline callout, interactive checklist with real-time percentage tracking, required documents list, and confusing jargon cards); (5) The interactive 'Ask FormFix AI' chat interface with 1-click suggested question chips; and (6) Dynamic import fallback allowing seamless execution whether files are in `services/` or the root directory.

services/document_processor.py (Document Ingestion & Normalization Engine):

The bridge between raw bytes and clean readable text. Responsibilities: (1) File extension routing (`.pdf` vs images); (2) Native PDF text extraction using `fitz.open(stream=file_bytes, filetype='pdf')`; (3) **Scanned Document Heuristic:** calculates the average text length per page across the PDF. If the average is under 40 characters per page, the PDF is flagged as a scanned image, and pages are converted into 150 DPI pixmaps for OCR; (4) `clean_whitespace()`: uses regular expressions to normalize multi-spaces, strip trailing line whitespace, and collapse consecutive empty lines down to a maximum of 2, minimizing LLM token consumption.

services/ocr_service.py (Optical Character Recognition Engine):

Specialized image preprocessing and OCR wrapper. Responsibilities: (1) `preprocess_image()`: takes a PIL Image, converts it to 8-bit grayscale (`L` mode), and applies an `ImageEnhance.Contrast` factor of 1.8 to sharpen faded text; (2) `_configure_tesseract()`: dynamically locates the Tesseract executable across system PATH and standard Windows install directories (`C:\Program Files\Tesseract-OCR\tesseract.exe`); (3) `extract_text_from_image()`: runs `pytesseract.image_to_string()` and returns clean UTF-8 text with user-friendly error fallbacks.

services/ai_service.py (Groq LLM Client & Prompt Engineering):

The artificial intelligence brain of FormFix AI. Responsibilities: (1) `get_groq_api_key()`: multi-tier credential lookup checking Streamlit Cloud Secrets (`st.secrets['GROQ_API_KEY']`), local `.env`, and OS environment variables; (2) `analyze_document_with_ai()`: formats the system prompt, enforces `response_format={'type': 'json_object'}`, dispatches to Groq's high-speed endpoint using `qwen/qwen3.8-27b`, and validates the 8 required JSON keys; (3) `ask_document_ai()`: handles interactive user questions strictly grounded in the document text; (4) `clean_json_response()`: strips any stray markdown code fences (` ```json `) to ensure safe JSON parsing.

requirements.txt: Minimal production dependencies (`streamlit`, `groq`, `python-dotenv`, `PyMuPDF`, `pytesseract`, `Pillow`).

.env & .env.example: Environment variable configuration containing `GROQ_API_KEY`. Protected from version control by `.gitignore`.

.gitignore: Ignores `.env`, compiled Python caches (`__pycache__/`, `*.pyc`), and local IDE files to ensure zero credential leaks.

test_formfix.py: Self-contained automated test suite executing 7 unit tests verifying PDF extraction, regex normalization, and schema conformance.

4. How Everything is Linked Together (Execution Flow)

The entire application operates as a cohesive, deterministic data pipeline. Here is the exact step-by-step lifecycle of an interaction:

Step 1: Upload & Byte Ingestion	The user drags a PDF or image into `st.file_uploader`. Streamlit places the raw file into an in-memory buffer (`uploaded_file.getvalue()`). The app displays the file name, MIME type, and calculated size in KB. Processing does not start automatically—waiting for explicit user intent.
Step 2: Trigger & Progress Activation	The user clicks **'Analyze Document'**. `app.py` triggers an animated multi-step progress bar: *File received → Extracting text → Understanding document → Identifying requirements → Creating action plan*.

Step 3: Text Extraction & Scanned Detection	`app.py` invokes `extract_document_text(file_name, file_bytes)` in `document_processor.py`. If PDF, PyMuPDF opens the stream. If page character density ≥ 40 chars/page, text is extracted directly. If average density < 40 chars/page, the document is flagged as scanned; PyMuPDF renders the page to a pixmap, and `ocr_service.py` runs contrast enhancement and Tesseract OCR. Extracted text is normalized by `clean_whitespace()`.
Step 4: AI Analysis & Schema Enforcement	`app.py` calls `analyze_document_with_ai(extracted_text)` in `ai_service.py`. The text is paired with the strict AI Document Action Assistant system prompt and dispatched to Groq's API (`qwen/qwen3.8-27b`) with `response_format={'type': 'json_object'}`. Groq returns a structured JSON string in under 2 seconds. `ai_service.py` parses the JSON, validates keys, and returns a clean dictionary.
Step 5: Dashboard & Stateful Action Plan	`app.py` saves the analysis in `st.session_state.analysis_result`. The action items list initializes a persistent dictionary `checklist_state = {item: False}`. The dashboard renders: (1) Interactive checklist with checkboxes and real-time completion percentage, (2) Prominent deadline callout, (3) Required certificates cards, and (4) Confusing fields decoded with supporting documents.
Step 6: User Interaction & Grounded Q&A;	When a user ticks a checkbox, Streamlit reactivity instantly recalculates progress (e.g. '3 of 7 tasks done - 42%'). If the user asks a question via the chat input (or clicks a suggested chip like 'What is the deadline?'), `app.py` calls `ask_document_ai()`. The LLM answers strictly using the document text, replying *'I couldn't find that information in the uploaded document'* if the query references out-of-scope facts.

5. Prompt Engineering & Anti-Hallucination Guardrails

To ensure complete reliability for academic and official documents, the AI prompt is engineered with strict operational constraints:

- **System Persona:** The model is instructed to act as an *AI Document Action Assistant* with the mission: Document → Understanding → Action.
- **Explicit Non-Hallucination Directive:** The prompt commands: *'Do not invent information. If information is not present in the document, explicitly say "Not specified" or use an empty list [].'*
- **Field Decoupling:** For every confusing field (e.g. 'Annual Family Income'), the model must provide both a simple English explanation and the exact supporting document required (e.g. 'Income Certificate').

```
{ "document_title": "Document title or Not specified", "document_type": "e.g. Scholarship Notice, Job Description,
Circular", "summary": "Concise 2-3 sentence overview", "deadline": "Specific deadline date/time or Not specified",
"required_documents": ["Document 1", "Document 2"], "important_fields": [{"field": "...", "explanation": "...",
"required_document": "..."}], "action_items": ["Step 1", "Step 2"], "warnings": ["Warning 1", "Warning 2"] }
```

6. Quality Assurance & Cloud Continuous Deployment

Automated Unit Tests (test_formfix.py): FormFix AI includes an automated test suite verifying: (1) In-memory digital PDF generation and extraction via PyMuPDF; (2) Whitespace regex normalization; (3) Unsupported file format rejection; (4) Empty document handling; (5) Pillow contrast preprocessing; (6) JSON schema conformance; and (7) Code-fence stripping. All 7 tests pass in 0.09s.

Continuous Cloud Deployment: The codebase is deployed to **Streamlit Community Cloud** (connected to GitHub repository paridhimuttyamwar/FormFix). Every Git push automatically triggers a fast cloud rebuild. API credentials are encrypted in Streamlit Cloud Secrets (st.secrets['GROQ_API_KEY']), ensuring zero key exposure on public version control.